FAST MARCHING METHOD



Departamento de Propulsión

Asociación de cohetería experimental de la Universidad de Sevilla

COHETEROS

1 Summary and target

1.1 Objectives

This document specifies how to calculate the geometry of a *finocyl* propellant grain and the quantities that depend on it, for incorporation into the HERMES simulator. The method replicates the one used by the OpenMotor software, which is based on the *Fast Marching Method* (FMM). The document is written so that the implementation can be carried out without consulting the original software code.

1.2 Summary of the method

The cross-section of the grain is represented as a binary image (propellant / void), where a value of 1 is assigned if there is propellant mass in that pixel and 0 otherwise. With FMM, the distance from each propellant pixel to the initial combustion surface is calculated. As the combustion front advances along its normal, the propellant consumed after a regression of d is the set of points at a distance less than d from that surface. From this distance map, calculated once, the cross-sectional area $A_{\text{face}}(d)$ and the burnt perimeter $P(d)$ are obtained, and from these, the combustion surface area, the propellant volume and the port area as a function of d. There are many hypotheses considered as uniform geometric regression normal to the surface, extruded 2D geometry and inert outer shell.

1.3 Target

- Definition of the Finocyl geometry with normal and inverted fins, and its validation.
- Calculation of the regression map using FMM.
- Grain functions: $S(d)$, $V(d)$, $A_{\text{port}}(d)$, web(d) and free volume
- Interface with the HERMES ballistic loop.
- A complete worked example with numerical values

2 The Fast Marching Method and its application to a grain

2.1 Problem statement

The cross-section of the grain is represented as a binary image in which each pixel is either void (core and fins, where no propellant is present) or propellant. Following the convention used by OpenMotor, the value 1 denotes propellant and the value 0 denotes void.

Combustion starts on the interface between void and propellant, which is the only part of the propellant exposed to the combustion gases. It is assumed that this burning surface recedes along its local normal direction at a speed $r(P)$, the burn rate, which depends on the chamber pressure P. The total regression distance after a time t is:

$$d(t) = \int_0^t r\big(P(\tau)\big)\,\mathrm{d}\tau. \tag{1}$$

Under this hypothesis, the propellant consumed after a regression d is exactly the set of points whose distance to the initial burning surface is smaller than d. Hence, how the cross-section burns is determined by one quantity: the distance from each propellant pixel to the void.

Distance to the void. Two sets of pixels are defined:

- Ω is the set of all propellant pixels.
- Γ is the set of all void pixels. Its boundary with Ω is the initial burning surface.

Let $p \in \Omega$ be a propellant pixel and $q \in \Gamma$ any void pixel. Then:

$$D(p) = \min_{q\in\Gamma} \|p - q\|, \tag{2}$$

where $\|p - q\|$ is the straight-line distance between p and q and the minimum keeps only the smallest of those distances. That is, $D(p)$ is the distance from p to the nearest void pixel. For example, if the void pixels closest to p are at 4, 7 and 12 mm, then $D(p) = 4$ mm.

Meaning of D. A pixel p is consumed when the regression reaches $d = D(p)$. In the example above, it burns when the surface has receded 4 mm. The values $D(p)$ of all pixels form a map of *burning times*, with d acting as a clock. Since D depends only on the geometry, it is computed once, before the ballistic simulation, and reused at every time step.

Direct evaluation. Equation (2) could be evaluated literally, measuring the distance from every propellant pixel to every void pixel, but the cost grows with the product of both numbers of pixels. The Fast Marching Method (FMM) obtains the same field by propagating values outwards from Γ, at a cost close to linear in the number of pixels.

2.2 Notation

Symbol	Meaning	Units
Ω	set of propellant pixels	–
Γ	set of void pixels (initial burning surface)	–
$D(p)$	distance from pixel p to Γ	m
h	pixel size (distance between neighbouring pixels)	m
d	regression distance of the burning surface	m
$r(P)$	burn rate at chamber pressure P	m/s
N	total number of pixels of the image	–
a, b	smallest known value of D among the two neighbours of a pixel along each axis	m
M^*	number of pixels per side of the square image (`mapDim`)	–
N_{px}	total number of pixels of the image, $N_{px} = M^2$	–
D_g, L	grain diameter and length	m
D_c, N_f	core diameter and number of fins	m, –
w, l	fin width and fin length	m
$\tilde{}$	quantity normalised with the grain radius, e.g. $\tilde{w} = 2w/D_g$	–
$A_{\text{face}}(d)$	propellant cross-sectional area	m^2
$P(d)$	burning perimeter of the cross-section	m

* M is the resolution of the image used to represent the cross-section (`mapDim` in OpenMotor). The image is a square of $M \times M$ pixels that covers the bounding square of the grain, so its side corresponds to the grain diameter D_g and the physical size of a pixel is $h \approx D_g/M$. The total number of pixels is $N_{px} = M^2$ (those outside the grain are masked). For example, with $D_g = 100$ mm and $M = 1001$, the pixel size is $h \approx 0.1$ mm. A larger M gives a smaller pixel and a more accurate result, but also a higher cost, which grows with M^2. OpenMotor uses $M = 1001$ by default and allows values between 250 and 2000.

Table 1: Notation used in this section.

2.3 Fundamentals

Propagation interpretation. The distance field D can be obtained by considering that the whole of Γ ignites simultaneously and that the combustion front propagates with unit speed. The instant at which the front reaches a pixel is equal to its distance D.

One-dimensional example. Consider a row of pixels with Γ on the left and $h = 1$:

void	p_1	p_2	p_3	p_4
0	?	?	?	?

The front reaches p_1 first, so $D(p_1) = 0 + h = 1$. The only path to p_2 goes through p_1, hence $D(p_2) = D(p_1) + h = 2$, and similarly for the remaining pixels. The value of a pixel is thus obtained from the value of the neighbour from which the front arrives, plus h, which requires the pixels to be processed in order.

Eikonal equation. A distance function increases at unit rate in the direction away from the set from which it is measured, which is expressed by the eikonal equation

$$|\nabla D| = 1 \quad \text{in } \Omega, \qquad D = 0 \quad \text{on } \Gamma. \tag{3}$$

The method does not solve this equation analytically: it approximates it on the pixel grid, using the values of neighbouring pixels. Moreover, the level curves $\{D = d\}$ coincide with the burning fronts of the grain after a regression d.

2.4 Algorithm overview

In two dimensions each pixel has four neighbours, so the value of a pixel depends on its neighbours and, at the same time, theirs depend on it. FMM resolves this apparent circularity using the following property: information propagates only from smaller to larger values of D. A pixel with a large value of D cannot influence a pixel with a smaller one. Consequently, if the pixels are processed in increasing order of D, all the pixels on which a given pixel depends have already been computed when its turn arrives.

Each pixel is assigned one of three states:

- **Known**: its value of D is final.
- **Narrow band**: it has a provisional value, as it is a neighbour of at least one Known pixel.
- **Far**: it has not yet been reached by the front.

The algorithm proceeds as follows:

1. All pixels of Γ are marked as Known with $D = 0$.
2. A provisional value is computed for each of their propellant neighbours, which are moved to the narrow band.
3. The pixel of the narrow band with the smallest value of D is marked as Known.
4. The values of its neighbours that are not Known are recomputed from the values of the neighbours that are already Known, and those that were Far are added to the narrow band.
5. Steps 3 and 4 are repeated until the narrow band is empty.

Computational cost. Each pixel becomes Known exactly once. If the narrow band is stored in a binary heap, selecting the smallest element has logarithmic cost, and the total cost is $O(N \log N)$.

2.5 Update formula

Step 4 of the algorithm requires a rule to compute the value D of a pixel from the values of its neighbours that are already Known. This rule is obtained by discretising the eikonal equation (3) on the pixel grid.

Discretisation. Let p be the pixel whose value D has to be computed and h the pixel size. In two dimensions the eikonal equation reads

$$\left(\frac{\partial D}{\partial x}\right)^2 + \left(\frac{\partial D}{\partial y}\right)^2 = 1. \tag{4}$$

Each partial derivative is approximated by a finite difference between p and one of its neighbours along the corresponding axis. Along each axis there are two candidates (left and right, down and up). Since the front travels from small to large values of D, it reaches p from the neighbour with the *smaller* value, so that is the one used. We define

$$a = \min(D_{\text{left}},\, D_{\text{right}}), \qquad b = \min(D_{\text{down}},\, D_{\text{up}}), \tag{5}$$

where a neighbour that is not Known yet is assigned the value $+\infty$, so that it is never selected. The derivatives are then approximated by

$$\frac{\partial D}{\partial x} \approx \frac{D-a}{h}, \qquad \frac{\partial D}{\partial y} \approx \frac{D-b}{h}. \tag{6}$$

This choice is called an *upwind* scheme: it only uses neighbours on the side from which the front arrives. A central difference would also use neighbours with larger values, which have not been computed yet.

Discrete equation. If a neighbour has a value larger than D, the front does not come from that side, and its contribution must be zero. This is expressed by keeping only the positive part of each difference:

$$\max\Big(\frac{D-a}{h},\, 0\Big)^2 + \max\Big(\frac{D-b}{h},\, 0\Big)^2 = 1. \tag{7}$$

The two terms can be interpreted as the components of the unit vector that gives the local direction of propagation. Equivalently, the equation assumes that, in the neighbourhood of p, the front is a straight line that moves with unit speed.

Equation (7) is solved for D. The result depends on how many of the two terms are active, which gives two cases. In both we assume $a \le b$; otherwise the roles of a and b are exchanged.

Case 1: $|a-b| \ge h$ (or $b = \infty$). Try a solution in which only the neighbour with value a contributes:

$$D = a + h = \min(a, b) + h. \tag{8}$$

Case 2: $|a-b| < h$**.** Now $a+h > b$, so the value $a+h$ of Case 1 would be larger than b. This means that the neighbour with value b also lies on the side from which thefront arrives, and both terms of Equation (7) must be kept. Removing the max and multiplying by h^2 gives

$$(D-a)^2 + (D-b)^2 = h^2 \quad \rightarrow \quad 2D^2 - 2(a+b)D + a^2 + b^2 - h^2 = 0. \tag{9}$$

Its two roots are

$$D_\pm = \frac{a+b \pm \sqrt{2h^2 - (a-b)^2}}{2}. \tag{10}$$

The square root is real because $|a-b| < h$ implies $2h^2 - (a-b)^2 > h^2 > 0$. Only one of the roots is valid. Keeping both terms requires $D > \max(a,b)$, since otherwise the max in Equation (7) would set one of them to zero. Writing $S = \sqrt{2h^2 - (a-b)^2}$, we have

$$D_+ - \max(a,b) = \frac{S - |a-b|}{2} > 0, \qquad D_- - \min(a,b) = \frac{|a-b| - S}{2} < 0, \tag{11}$$

because $S > h > |a-b|$. Hence D_+ satisfies the condition and D_- lies below both neighbours, which contradicts the assumption. The update formula is therefore

$$D = \frac{a+b+\sqrt{2h^2 - (a-b)^2}}{2}. \tag{12}$$

Consistency of the two cases. For $|a-b| = h$ (say $b = a+h$), the formula of Case 2 gives $D = (2a+h+h)/2 = a+h$, which coincides with Case 1. The update is therefore continuous in a and b.

Examples. For $a=1$, $b=3$ and $h=1$, we have $|a-b| = 2 \geq h$, so Case 1 applies and $D=2$. For $a=b=1$ and $h=1$, Case 2 applies:

$$D = \frac{1+1+\sqrt{2}}{2} \approx 1.707. \tag{13}$$

Substituting into Equation (7), $(0.707)^2 + (0.707)^2 = 1$: the front arrives along the diagonal, with direction components (0.707, 0.707). Using only the rule of Case 1 would give $D=2$, and the fronts would be squares instead of circles.

Geometric meaning. Case 2 assumes that the front is a straight line passing through the two neighbours. The value of x is then the value of that line plus the perpendicular distance from x to it. For example, with $a=b=1$ and $h=1$ the line is at distance $1/\sqrt{2} \approx 0.707$ from x, and the formula gives $D = (1+1+\sqrt{2})/2 \approx 1.707$. Using only the rule of Case 1 would give 2 and the fronts would be square instead of circular.

Remark on the order of the scheme. The formulas above correspond to a first-order scheme. The library `scikit-fmm`, which OpenMotor uses, applies by default a second-order stencil (`order=2`) when enough Known neighbours are available. The principle is the same, but the values differ slightly. Therefore, a reimplementation of the update formula will not reproduce OpenMotor's numbers exactly; see Section **??**.

2.6 Worked example

Consider a 5×5 image with $h = 1$ in which only the central pixel is void. The rest of the pixels are propellant.

1. The central pixel is Known with $D = 0$.
2. Its four direct neighbours are updated with Case 1: $D = 0 + 1 = 1$. They are the smallest values of the narrow band, so they become Known.
3. Each diagonal neighbour first receives a provisional value 2 (Case 1, from the first axis neighbour that becomes Known). When the second one becomes Known, it is recomputed with Case 2, with $a = b = 1$: $D = (1 + 1 + \sqrt{2})/2 = 1.707$.
4. The pixels two positions away along an axis only have one Known neighbour: $D = 1 + 1 = 2$.
5. The pixels at $(2, 1)$ have $a = 2$ and $b = 1.707$. Since $|a - b| = 0.293 < 1$, Case 2 gives $D = (2 + 1.707 + \sqrt{2 - 0.293^2})/2 = 2.545$.
6. Each corner has $a = b = 2.545$, so $D = (2.545 + 2.545 + \sqrt{2})/2 = 3.252$.

Note that the values become Known in increasing order (1, 1.707, 2, 2.545, 3.252), as the algorithm requires.

FMM values:

3.25	2.55	2.00	2.55	3.25
2.55	1.71	1.00	1.71	2.55
2.00	1.00	0.00	1.00	2.00
2.55	1.71	1.00	1.71	2.55
3.25	2.55	2.00	2.55	3.25

Exact Euclidean distance:

2.83	2.24	2.00	2.24	2.83
2.24	1.41	1.00	1.41	2.24
2.00	1.00	0.00	1.00	2.00
2.24	1.41	1.00	1.41	2.24
2.83	2.24	2.00	2.24	2.83

Along the axes both coincide. Elsewhere the first-order FMM overestimates the distance by up to about 0.4 pixels. This is an unfavourable case, since the source is a single pixel; with long and smooth fronts, such as a circular core, the error is smaller.

From the map to the cross-sectional area. The area of propellant left after a regression d is the number of pixels with $D > d$ multiplied by the pixel area h^2:

d	0.5	1.5	2.2	3.0	≥ 3.25
pixels with $D > d$	24	20	12	4	0

The maximum value of the map, 3.25, is the maximum web: the regression at which all the propellant has burned.

2.7 Application to the finocyl grain

The previous subsections explain FMM for a generic image. Here it is applied to the finocyl grain, following the same procedure as OpenMotor. The goal is to obtain two functions of the regression distance d: the propellant cross-sectional area $A_{\text{face}}(d)$ and the burning perimeter $P(d)$. The procedure has four steps:

1. Draw the cross-section of the grain as a binary image (propellant = 1, void = 0).
2. Apply FMM to the image to obtain the distance map.
3. Obtain $A_{\text{face}}(d)$ by counting the pixels that have not burned yet.
4. Obtain $P(d)$ by measuring the length of the burning front.

The finocyl geometry. A finocyl grain has a circular core with rectangular extensions, called fins, that start at the core wall and run along its normal direction. In a normal finocyl the fins are slots cut outwards into the propellant. In an inverted finocyl the fins are ribs of propellant that protrude inwards, into the core. The geometry is defined by the following parameters (Figure 1): the grain diameter D_g, the core diameter D_c, the number of fins N_f, the fin width w and the fin length l (measured from the core wall), plus a flag that selects normal or inverted fins. The fins are equally spaced around the core.

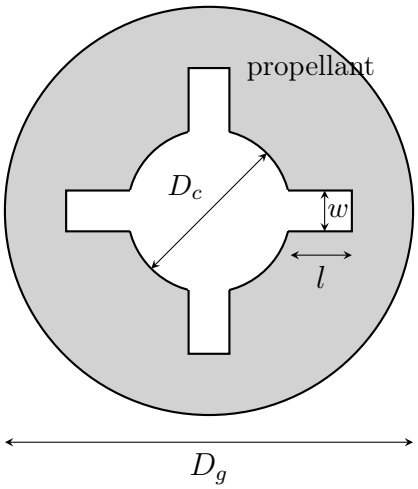


Figure 1: Cross-section of a finocyl grain with normal fins ($N_f = 4$). Grey: propellant. White: void (core and fins).

Step 1a: image and normalised coordinates. The image has $M \times M$ pixels and covers the square $[-1, 1] \times [-1, 1]$, with the origin at the centre of the grain. In these normalised coordinates the grain radius is 1, so a pixel at (x, y) is inside the grain if $x^2+y^2 \leq 1$. The pixels outside (the corners of the square) are masked: they are excluded from all later calculations.

All the lengths of the grain are expressed as fractions of the grain radius $D_g/2$:

$$\tilde{r}_c = \frac{D_c}{D_g}, \qquad \tilde{w} = \frac{2w}{D_g}, \qquad \tilde{l} = \frac{2l}{D_g}. \tag{14}$$

Here $\tilde{r}_c$ is the core *radius*. For the example grain ($D_g = 100$ mm, $D_c = 40$ mm, $w = 10$ mm, $l = 15$ mm), we get $\tilde{r}_c = 0.4$, $\tilde{w} = 0.2$ and $\tilde{l} = 0.3$. A normalised length of 0.4 simply means 40 % of the grain radius.

Step 1b: draw the core. The map starts with all pixels equal to 1 (propellant). The core is drawn first: every pixel whose distance to the centre is smaller than the core radius is set to 0,

$$\sqrt{x^2 + y^2} < \tilde{r}_c \implies \text{pixel} = 0. \tag{15}$$

Step 1c: draw the fins. Each fin is a rectangle. To describe it, it is easier to use a frame of reference rotated towards the fin. Fin number $i = 0, \ldots, N_f - 1$ is rotated by the angle

$$\theta_i = \frac{2\pi\, i}{N_f}. \tag{16}$$

In the rotated frame of fin i, a pixel (x, y) has two coordinates:

$$s = x \sin\theta_i - y \cos\theta_i, \qquad t = x \cos\theta_i + y \sin\theta_i. \tag{17}$$

Here s is the position along the fin, measured from the centre of the grain, and t is the position across the fin, measured from the fin axis. These two expressions are just the usual change of coordinates for a rotation of the axes. The pixels of fin i are those inside the rectangle

$$|t| < \frac{\tilde{w}}{2} \qquad \text{and} \qquad s_{\text{start}} < s < s_{\text{end}}, \tag{18}$$

that is, a strip of width $\tilde{w}$ centred on the fin axis, closed at both ends. The limits s_{start}, s_{end} and the value assigned to those pixels depend on the type of fin:

Fin type	s_{start}	s_{end}	Value assigned
Normal	0	$\tilde{r}_c + \tilde{l}$	0 (propellant removed)
Inverted	$\tilde{r}_c - \tilde{l}$	$\tilde{r}_c$	1 (propellant added)

For normal fins the rectangle starts at the centre and ends a distance $\tilde{l}$ beyond the core wall; the part inside the core was already 0, so only the part that sticks out of the core changes the map. For inverted fins the rectangle covers the last $\tilde{l}$ before the core wall, and it is filled with propellant again. This is why the core is drawn before the fins: the fins of an inverted finocyl have to overwrite the void of the core.

Numerical example. For fin $i = 0$ ($\theta_0 = 0$), $s = -y$ and $t = x$. With the example grain (normal fins, s from 0 to 0.7), the pixels of the fin satisfy $|x| < 0.1$ and $0 < -y < 0.7$: a rectangle below the centre. The pixel (0.05, −0.5) belongs to the fin and is set to 0, while (0.2, −0.5) is outside ($|x| > 0.1$) and remains propellant. For $i = 1$ ($\theta_1 = 90°$) we have $s = x$ and $t = y$, so the fin points along $+x$. The four fins point along $-y$, $+x$, $+y$ and $-x$. If the fins were inverted, the same rectangle would span $s \in (0.1, 0.4)$ and would be set to 1.

Remark on the orientation. The first fin ($i = 0$) points along $-y$, and not along $+x$ as the angle $\theta_0 = 0$ might suggest. This is how OpenMotor defines it. Since the fins are equally spaced, this only rotates the whole pattern; the area, the perimeter and the web do not change.

Step 2: distance map. FMM is applied to the binary map. The void pixels (0) are the starting set Γ, and the masked pixels are excluded. The result is, for each propellant pixel, the distance to the nearest void pixel. We denote it $\tilde{D}(p)$, and it is expressed as a fraction of the grain radius.

Units. FMM returns distances in the units of the pixel size that it is given. In normalised coordinates the image covers the interval from −1 to 1, that is, a length of 2 shared among M pixels, so one pixel measures approximately $2/M$. Therefore:

- If n pixels are counted between two points, their distance is $\tilde{D} = n \cdot 2/M$

- To convert it to physical units, $\tilde{D}$ is multiplied by the real grain radius:

$$D(p) = \tilde{D}(p)\,\frac{D_g}{2}. \tag{19}$$

For the example grain ($D_g = 100$ mm), $\tilde{D} = 0.3$ corresponds to 15 mm.

The largest value of the map is the maximum web, the regression at which the last propellant burns:

$$w_{\max} = \frac{D_g}{2}\,\max_p \tilde{D}(p). \tag{20}$$

Since the masked pixels do not take part in the calculation, the outer casing does not generate any burning front: the grain burns only from the void (core and fins) towards the casing, and its outer surface is inhibited.

Implementation note. OpenMotor calls `skfmm.distance` on the masked map with a cell size `dx`$= 1/M$ and multiplies the result by 2. This is equivalent to using a cell size of $2/M$, and it gives $\tilde{D}$ directly in normalised units.

Step 3: cross-sectional area $A_{\text{face}}(d)$. The question is how much propellant is left after the surface has receded d. By the definition of D, the pixels that have not burned yet are those with $D > d$. Therefore the area is the number of those pixels multiplied by the area of one pixel:

$$A_{\text{face}}(d) = \underbrace{\#\{\,p:\ \tilde{D}(p) > \tilde{d}\,\}}_{\text{pixels not burned}} \times \underbrace{\left(\frac{D_g}{M}\right)^2}_{\text{pixel area}}, \qquad \tilde{d} = \frac{2d}{D_g}. \tag{21}$$

Only pixels inside the grain are counted. For $D_g = 100$ mm and $M = 1001$ the pixel size is about 0.1 mm and its area about 0.01 mm^2. As a sanity check, a grain with a core and no fins must give $A_{\text{face}}(0) \approx \pi(D_g^2 - D_c^2)/4$.

The area is not computed on the fly for every d of the simulation. It is computed once, for a table of regression values

$$\tilde{d}_k = \frac{k}{M}, \qquad k = 0, 1, 2, \ldots, K-1 \tag{22}$$

where K is the first index for which $\tilde{d}_k$ exceeds the maximum of the map (from there on there is no propellant left). Since one pixel measures $2/M$, the table has two samples per pixel. For each $\tilde{d}_k$ we count the pixels with $\tilde{D} > \tilde{d}_k$ and multiply by the area of a pixel, which gives a list of pairs $(\tilde{d}_k, A_k)$.

Smoothing. Counting whole pixels makes the table slightly irregular: each time $\tilde{d}_k$ goes past the value of a pixel, the count drops by one, and on a square grid these drops are not evenly distributed along a circular front. The jumps are small (one pixel is about 0.01 mm^2 in the example grain), but the simulation obtains the mass that burns in each time step as a

difference of volumes between consecutive steps, and differences amplify this kind of noise. To remove it, the table is filtered with a Savitzky-Golay filter (window of 31 samples, polynomial of order 5). For each sample, the filter takes the 31 samples centred on it (15 on each side), fits a polynomial of order 5 to them by least squares, and replaces the sample by the value of that polynomial at the centre. It is similar to a moving average, which is a polynomial of order 0, but since the polynomial has order 5 it follows the curvature of the curve instead of flattening it. The window covers about 15 pixels (≈ 1.5 mm in the example). On that scale $A_{\text{face}}(d)$ changes smoothly, so the polynomial follows it, while the one-pixel noise cannot be followed and is averaged out.

Limitations. Near the two ends of the table the window cannot be centred and the filter is less reliable. Besides, the table needs at least 31 samples, that is, $\tilde{D}_{\max} M \gtrsim 29$ (a web thinner than about 6 mm for $M = 250$ and $D_g = 100$ mm); SciPy's default mode is expected to give an error for a shorter table. **[Decision HERMES]**: reduce the window for thin webs.

Interpolation. The filtered table is interpolated linearly, so that $A_{\text{face}}(d)$ can be evaluated at any d (with $\tilde{d} = 2d/D_g$). For a regression beyond the end of the table, $A_{\text{face}} = 0$.

Step 4: burning perimeter $P(d)$. After a regression d, the burning front is the line that separates burned from unburned propellant: the contour curve $\tilde{D} = \tilde{d}$ of the distance map. Its length is the burning perimeter. This curve runs between the pixels, so it cannot be measured by counting them. Counting the boundary pixels gives the length of a staircase, which for a circle is about $4/\pi \approx 1.27$ times too large, and the error does not decrease when M increases.

OpenMotor measures it with a *marching squares* algorithm (function `find_perimeter`, modelled on `find_contours` of scikit-image). The idea is to cut the contour into small straight segments, one per block of 2×2 pixels, and to add up their lengths. The procedure for each block is the following.

1. **Classify the corners.** Let $\tilde{D}_{UL}$, $\tilde{D}_{UR}$, $\tilde{D}_{LL}$ and $\tilde{D}_{LR}$ be the values of the map at the upper left, upper right, lower left and lower right corners of the block. A corner is "above" the level if its value is strictly larger than $\tilde{d}$ (a corner equal to the level counts as below). The four answers are combined into a case number from 0 to 15:

$$\text{case} = [\tilde{D}_{UL} > \tilde{d}] + 2\,[\tilde{D}_{UR} > \tilde{d}] + 4\,[\tilde{D}_{LL} > \tilde{d}] + 8\,[\tilde{D}_{LR} > \tilde{d}], \tag{23}$$

   where $[\,\cdot\,]$ is 1 if the condition holds and 0 otherwise. If the four corners are on the same side (cases 0 and 15), the front does not cross the block and nothing is added.

2. **Locate the crossings.** If the corners are not all on the same side, the front crosses some edges of the block. On an edge that joins two corners with values a and b, the position where $\tilde{D}$ equals the level is found by linear interpolation:

$$f = \frac{\tilde{d} - a}{b - a}, \tag{24}$$

which is the fraction of the edge, measured from the corner with value a, at which the crossing lies (if $a = b$, the code takes $f = 0$ to avoid dividing by zero). Four fractions are defined, one per edge: f_T (top edge, from the upper left corner), f_B (bottom edge, from the lower left corner), f_L (left edge, from the lower left corner) and f_R (right edge, from the lower right corner).

3. **Join the crossings.** The two crossing points of the block are joined by a straight segment. Its length is the hypotenuse of the horizontal and vertical distances between both points, which depends on the case. For example, in case 1 (only the upper left corner above the level) the front goes from the top edge to the left edge, at a distance f_T from the left side and $1 - f_L$ from the top side, so

$$\ell_1 = \sqrt{f_T^2 + (1 - f_L)^2} \text{ pixels.} \tag{25}$$

The lengths of the 14 cases are listed in Annex A. In cases 6 and 9, with two opposite corners above the level, the block is ambiguous and contains two segments, one for each corner.

The perimeter in pixels, P_{px}, is the sum of the lengths of the segments of all the blocks, and it is converted to metres with the size of a pixel:

$$P(d) = \frac{D_g}{M} P_{\text{px}}(\tilde{d}), \qquad \tilde{d} = \frac{2d}{D_g}. \tag{26}$$

The level is $\tilde{d}$ because the distance map is in normalised units.

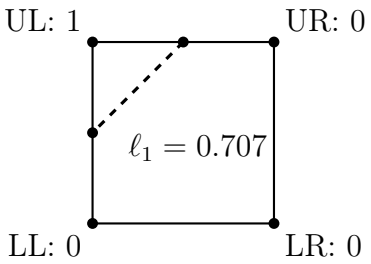


Figure 2: Block with one corner above the level ($\tilde{d} = 0.5$): case 1. The front crosses the top and left edges at their midpoints.

Example. Take the block of Figure 2: the upper left corner has value 1, the other three have value 0, and the level is 0.5. This is case 1. The interpolation along the top edge gives $f_T = (0.5 - 1)/(0 - 1) = 0.5$, and along the left edge (from the lower left corner, with value 0, to the upper left one, with value 1) $f_L = (0.5 - 0)/(1 - 0) = 0.5$. Then $\ell_1 = \sqrt{0.5^2 + (1 - 0.5)^2} = 0.707$ pixels: the diagonal of a square of side 0.5. A block taken from a real grain is worked out in Section 4.4.

What the algorithm does not do.

- It does not join the segments into closed curves to measure them. That is only done when the contours are needed to draw them; the length is already added up block by block.
- It does not smooth anything. The contour is a polyline inscribed in the real curve, so it slightly underestimates its length. The error is small, because each segment is about one pixel long.
- The option `fully_connected` does not change the length. In the ambiguous cases 6 and 9 the code always cuts off each corner above the level separately; the option only changes how the segments are joined into curves for drawing.

What is left out of the count.

- The first and last three rows and columns of the image are not scanned.
- Blocks whose centre is farther than $M/2 - 3$ pixels from the centre of the image are ignored, which excludes a ring of about three pixels next to the casing. The masked pixels hold the value 0 in the map, below any positive level, so without this exclusion a false contour would appear along the casing. The consequence is that the front is not measured during the last ≈ 3 pixels of regression before it reaches the casing (≈ 0.3 mm for $D_g = 100$ mm and $M = 1001$; our estimate).

Remarks.

- Corners equal to the level count as below it. At $d = 0$ the level is 0, the value of the void pixels, so the contour runs through the centres of the boundary void pixels and its length is that of a staircase through them. In our tests, this overestimates $P(0)$ by about 5 % for a circular core and by 1.6 % for the finocyl of the worked example, and the error falls quickly as d grows (Annex B).
- **[Decision HERMES]** Any routine that sums the lengths of the marching squares segments is valid. Two options for the region next to the casing: reproduce the exclusion of OpenMotor, or ignore only the blocks that touch a masked pixel.

3 Parameters, outputs and geometry validation

3.1 Input parameters

Table 2 lists what the finocyl grain needs. Names and ranges are those of the OpenMotor input fields; HERMES can use its own as long as the meaning is the same. The last two rows are settings of the motor, not of the grain: the grain just receives them.

Parameter	Symbol	Units	Range in OpenMotor
Grain diameter	D_g	m	0 to `maximumRefDiameter`*
Grain length	L	m	0 to `maximumRefLength`*
Inhibited ends	–	–	Neither, Top, Bottom, Both
Core diameter	D_c	m	0 to `maximumRefDiameter`*
Number of fins	N_f	–	integer, 0 to 64
Fin width	w	m	0 to `maximumRefDiameter`*
Fin length (from the core wall)	l	m	0 to `maximumRefDiameter`*
Inverted fins	–	–	true / false
Map dimension (resolution)	M	–	integer, 250 to 2000
Burnout web threshold	ε_w	m	$2.54 \cdot 10^{-5}$ to $3.175 \cdot 10^{-3}$

* These constants are defined in OpenMotor's `constants.py`.

Table 2: Input parameters of the finocyl grain.

Two remarks. With $N_f = 0$ the grain is a simple circular core. And the code refuses $M < 64$, because the map would be too coarse to give meaningful results.

3.2 Outputs

For the simulator the grain is a black box: HERMES gives it a regression distance d and it returns geometry. It works in two phases.

Setup (once per set of parameters). It draws the cross-section, applies FMM and builds the table of A_{face} (Section 2.7). It produces the maximum web $w_{\max}$, the table of areas and the means to evaluate the burning perimeter. If any parameter changes, the setup has to be repeated.

Queries (at every time step). The functions of Table 3, which are derived in Section 4 from A_{face}, P, $w_{\max}$ and the dimensions of the grain.

Function	Symbol	Units
Cross-sectional area of propellant	$A_{\text{face}}(d)$	m^2
Burning perimeter of the cross-section	$P(d)$	m
Regressed length of the grain	$L(d)$	m
Burning surface	$S(d)$	m^2
Propellant volume	$V(d)$	m^3
Port area	$A_{\text{port}}(d)$	m^2
Web left	$\text{web}(d)$	m
Free volume	$V_{\text{free}}(d)$	m^3

Table 3: Functions that the grain provides to the simulator.

3.3 How the simulator uses them

This is the loop of OpenMotor's `motor.py`, simplified. The grain only provides geometry; the ballistic model of HERMES (burn rate, pressure, thrust) is not part of this work.

```
for each time step:
    for each grain g:
        if web_g(d_g) > eps_w:           # grain still burning
            d_g += dt * r(P)             # r(P): burn rate of the propellant
    S_total = sum of S_g(d_g) over grains with web_g(d_g) > eps_w
    Kn      = S_total / A_throat
    P       = pressure from Kn           # propellant model of HERMES
    ... thrust, propellant mass, etc.
```

A grain whose web is below ε_w is considered burnt out and contributes no burning surface. The grain must not depend on pressure or time: only on d.

3.4 Geometry validation

Before simulating, the grain checks its parameters. *Errors* prevent the simulation; *warnings* only notify the user of a probable mistake. With $R_c = D_c/2$, OpenMotor applies the checks of Table 4.

#	Level	Condition	OpenMotor message
1	Error	$D_g = 0$	Diameter must not be 0
2	Error	$L = 0$	Length must not be 0
3	Error	$D_c = 0$	Core diameter must not be 0
4	Error	$D_c > D_g$	Core diameter must be less than or equal to grain diameter
5	Error	$l = 0$	Fin length must not be 0
6	Error	$w = 0$	Fin width must not be 0
7	Warning	$2l > D_g$	Fin length should be less than or equal to grain radius
8	Warning	normal fins and $D_c + 2l > D_g$	Core radius plus fin length should be less than or equal to grain radius
9	Error	inverted fins, $N_f > 0$, $l > R_c$ and $\sqrt{(l - R_c)^2 + (w/2)^2} > R_c$	Fin tips outside of core
10	Warning (normal) / Error (inverted)	$N_f > 1$ and $2a \tan(\pi/N_f) < w$, with $a = R_c + l$ (normal) or $a = R_c - l$ (inverted)	Fin tips intersect

Table 4: Geometry checks of the finocyl grain (rows 1 and 2 belong to the base grain class).

What check 10 means. The fin ends lie on a circle of radius a. Neighbouring fins are $2\pi/N_f$ apart, and the chord that joins their centres is $2a \sin(\pi/N_f)$. OpenMotor uses $2a \tan(\pi/N_f)$, the side of the polygon that circumscribes that circle, and compares it with the fin width: if the width is larger, neighbouring fins touch. For inverted fins this would split the core, hence the error.

Remark: For $N_f = 2$, $\tan(\pi/2)$ is infinite and check 10 never triggers, for $N_f = 1$ it is not evaluated

4 Derived quantities

4.1 Why the grain is a prism

So far we only have a 2D map. To get the 3D grain we need one idea: the distance from a point to a union of sets is the smallest of the distances to each set. The burning surface of a perforated grain starts on two kinds of void: the core and fins (a cross-section that runs along the whole length) and the outside of each exposed end face. A point of the grain at a depth z from an exposed end and at a distance $D(p)$ from the cross-section void therefore burns when the regression reaches $\min\big(D(p),\, z\big)$.

In words, the radial fronts (from core and fins) and the axial fronts (from the ends) advance independently. After a regression d the propellant that is left is a **prism**: the cross-section $\{\tilde{D} > \tilde{d}\}$, with area $A_{\text{face}}(d)$, extruded along a length $L(d)$. This is exact inside the model of uniform regression, not an extra approximation.

4.2 Ends of the grain

Let z be the position along the axis measured from the head end ($z = 0$) of the unburned grain. An end that is inhibited does not regress; an exposed end moves d into the grain.

Inhibited ends	$z_{\text{top}}(d)$	$z_{\text{bot}}(d)$	Exposed faces n
Neither	d	$L - d$	2
Top	0	$L - d$	1
Bottom	d	L	1
Both	0	L	0

Table 5: Position of the ends of the grain.

The regressed length is $L(d) = z_{\text{bot}}(d) - z_{\text{top}}(d)$.

The regression law does not depend on the direction: every exposed surface recedes along its own normal at the same speed $r(P)$, which is a property of the propellant and of the pressure. The walls of the core and the fins have their normals in the x–y plane, so they recede radially, and this is what the distance map describes. The end faces have their normals along the axis, so they recede along z: the face at the head end moves from $z = 0$ to $z = d$, and the face at the nozzle end from $z = L$ to $z = L - d$. There is a single regression d, applied to each surface along its normal. The model neglects the effects that make the burn rate depend on the position, such as erosive burning.

4.3 Formulas

$$S(d) = \underbrace{P(d)\,L(d)}_{\text{core and fins}} + \underbrace{n\,A_{\text{face}}(d)}_{\text{exposed end faces}} \tag{27}$$

$$V(d) = A_{\text{face}}(d)\,L(d) \tag{28}$$

$$A_{\text{port}}(d) = \frac{\pi D_g^2}{4} - A_{\text{face}}(d) \tag{29}$$

$$\text{web}(d) = \begin{cases} w_{\max} - d & \text{if both ends are inhibited,} \\ \min\big(L(d),\ w_{\max} - d\big) & \text{otherwise,} \end{cases} \tag{30}$$

$$V_{\text{free}}(d) = \frac{\pi D_g^2}{4}\,L - V(d) \tag{31}$$

- S: the lateral surface of a prism is its perimeter times its length; then we add the exposed end faces.
- V: volume of the prism.
- A_{port}: area of the casing tube minus the propellant, that is, the area of the void through which the gas flows.
- web: how much regression is left before the propellant is gone, radially ($w_{\max} - d$) or axially ($L(d)$), whichever runs out first. If both ends are inhibited, only the radial burn is possible.
- V_{free}: volume of the bounding cylinder not occupied by propellant, with $\pi D_g^2 L/4$ as the volume of that cylinder.

4.4 Worked example: a complete finocyl

This example follows the whole procedure for one grain, from the parameters to the quantities that HERMES needs. The numbers have been computed with a Python implementation of the pseudocode of Annex A (not with OpenMotor itself), with $M = 1001$, so small differences with OpenMotor are possible.

Data.

Parameter	Value
Grain diameter D_g	100 mm
Grain length L	150 mm
Core diameter D_c	40 mm
Number of fins N_f	4
Fin width w	10 mm
Fin length l	15 mm
Fins	normal
Inhibited ends	neither
Map dimension M	1001

Step 0: geometry checks. None of the checks of Table 4 is triggered: $D_c = 40 < D_g = 100$ mm, $2l = 30 \le 100$ mm, $D_c + 2l = 70 \le 100$ mm and, with $a = R_c + l = 35$ mm, $2a\tan(\pi/4) = 70 \ge w = 10$ mm, so neighbouring fins do not touch.

Step 1: normalised lengths and image.

$$\tilde{r}_c = \frac{40}{100} = 0.4, \qquad \tilde{w} = \frac{2 \cdot 10}{100} = 0.2, \qquad \tilde{l} = \frac{2 \cdot 15}{100} = 0.3. \tag{32}$$

The fins are normal, so $s_{\text{start}} = 0$ and $s_{\text{end}} = 0.4 + 0.3 = 0.7$, and each fin is the strip $|t| < 0.1$. The four fins point along $-y$, $+x$, $+y$ and $-x$ (fins $i = 0, 1, 2, 3$). For fin 0 ($\theta_0 = 0$) we have $s = -y$ and $t = x$: the pixel (0.05, −0.5) has $|t| = 0.05 < 0.1$ and $s = 0.5 \in (0, 0.7)$, so it is void (0), while (0.2, −0.5) has $|t| = 0.2 > 0.1$ and stays propellant (1).

The image has $1001^2 = 1\,002\,001$ pixels. Each pixel measures $D_g/M = 0.0999$ mm and has an area of 0.00998 mm^2. Of the total, 216 656 pixels are masked (outside the grain), and of the 785 345 inside, 186 087 are void and 599 258 are propellant.

Step 2: distance map. FMM gives $\tilde{D}$ for the 599 258 propellant pixels. Table 6 compares four pixels with a hand estimate.

Pixel (x, y)	Nearest void	Hand estimate	FMM
(0, −0.8)	tip of fin 0 ($y = -0.7$)	$0.1 \cdot 50 = 5.0$ mm	5.04 mm
(0, 0.75)	tip of fin 2 ($y = 0.7$)	$0.05 \cdot 50 = 2.5$ mm	2.55 mm
(0.45, 0.45)	core wall	$(\sqrt{0.45^2 + 0.45^2} - 0.4) \cdot 50 = 11.8$ mm	11.83 mm
(0.70, 0.70)	core wall	$(\sqrt{0.70^2 + 0.70^2} - 0.4) \cdot 50 = 29.5$ mm	29.49 mm

Table 6: Distance map: four pixels checked by hand.

The differences are about half a pixel (0.05 mm), as expected, since FMM measures from the centres of the void pixels. The largest value of the map is $\tilde{D}_{\max} = 0.5997$, found at the casing between two fins, so

$$w_{\max} = 0.5997 \cdot \frac{100}{2} = 29.99 \text{ mm}. \tag{33}$$

By hand: a point at radius r is at most $r - R_c$ from the core, hence $w_{\max} \le R - R_c = 30$ mm, and the points of the casing at 45° between fins reach it (their distance to the nearest fin corner is ≈ 30.4 mm).

Step 3: table of areas. The table has $K = \text{int}(0.5997 \cdot 1001) + 2 = 602$ samples ($k = 0, \ldots, 601$), with $\tilde{d}_k = k/1001$. For $k = 0$, the area is $599\,258 \cdot 0.00998 = 5980.6$ mm^2. Table 7 shows some samples before and after the Savitzky–Golay filter.

k	$\tilde{d}_k$	d (mm)	A_k raw (mm^2)	A_k filtered (mm^2)
0	0	0	5980.6	5983.6
1	0.0010	0.05	5976.4	5971.6
2	0.0020	0.10	5956.9	5959.4
100	0.0999	5.00	4674.3	4674.8
300	0.2997	15.0	1690.3	1691.2
600	0.5994	30.0	0.29	0.23

Table 7: Table of areas: raw and filtered values.

In the interior the filter changes the values by less than 1 mm^2 (0.02 %); at the start of the table the change is larger (up to a few mm^2), as expected from a filter whose window is not centred there.

Step 4: perimeter. Take $d = 5$ mm, that is, $\tilde{d} = 0.1$. One of the blocks crossed by the front lies along the straight side of fin 0, at $x \approx 0.198$, $y = -0.55$. Its corners have $\tilde{D} = 0.0989$ (upper left), 0.1009 (upper right), 0.0989 (lower left) and 0.1009 (lower right). The two right corners are above the level, so it is case 10. The crossing on the top edge is at

$$f_T = \frac{0.1 - 0.0989}{0.1009 - 0.0989} = 0.55, \tag{34}$$

and the same on the bottom edge, $f_B = 0.55$. The segment length is $h(f_T - f_B,\, 1) = \sqrt{0 + 1} = 1$ pixel: a vertical segment, as it should be on the side of a fin. The total P_{px} is the sum over all the blocks of the image. Converted with D_g/M, it gives $P(5 \text{ mm}) = 273.3$ mm.

At $d = 0$ the code gives $P(0) = 254.3$ mm. The hand estimate is

$$P(0) \approx \pi D_c + N_f \left[2\left(R_c + l - \sqrt{R_c^2 - w^2/4}\right) + w - 2R_c \arcsin \frac{w}{2R_c} \right] = 125.7 + 4 \cdot 31.16 = 250.3 \text{ mm}. \tag{35}$$

The code is 1.6 % higher; the reason is explained in Annex B.

Step 5: derived quantities. For $d = 10$ mm: $A_{\text{face}} = 3241.1$ mm^2, $P = 299.6$ mm and $L(d) = 150 - 2 \cdot 10 = 130$ mm. Then

$$S = 299.6 \cdot 130 + 2 \cdot 3241.1 \approx 45\,430 \text{ mm}^2, \tag{36}$$

$$V = 3241.1 \cdot 130 \approx 421\,343 \text{ mm}^3, \tag{37}$$

$$A_{\text{port}} = \tfrac{\pi}{4}\, 100^2 - 3241.1 = 4612.9 \text{ mm}^2, \tag{38}$$

$$\text{web} = \min(130,\ 29.99 - 10) = 19.99 \text{ mm}, \tag{39}$$

$$V_{\text{free}} = \tfrac{\pi}{4}\, 100^2 \cdot 150 - 421\,343 \approx 756\,755 \text{ mm}^3. \tag{40}$$

Table 8 gives the same quantities for other values of d. At $d = 0$ the propellant volume is 897 534 mm^3, a volume loading of 76.2 % of the 1 178 097 mm^3 of the bounding cylinder.

d (mm)	A_{face} (mm^2)	P (mm)	$L(d)$ (mm)	S (mm^2)	V (mm^3)	A_{port} (mm^2)	web (mm)	V_{free} (mm^3)
0	5983.6	254.3	150	50 105	897 534	1870.4	29.99	280 564
5	4673.5	273.3	140	47 613	654 283	3180.5	24.99	523 815
10	3241.1	299.6	130	45 430	421 343	4612.9	19.99	756 755
15	1687.4	231.8	120	31 194	202 487	6166.6	14.99	975 611
20	752.3	142.4	110	17 170	82 752	7101.7	9.99	1 095 345
25	199.6	72.3	100	7 631	19 958	7654.4	4.99	1 158 140

Table 8: Derived quantities of the example grain.

Step 6: use in the simulation loop. To see how HERMES would use the grain, suppose a constant burn rate $r = 6$ mm/s, a time step $\Delta t = 0.5$ s and a throat of 20 mm in diameter ($A_t = 314.16$ mm^2). These values only make the numbers easy to follow; the real r depends on the pressure.

Step	d (mm)	A_{face} (mm^2)	P (mm)	S (mm^2)	$K_n = S/A_t$
0	0	5983.6	254.2	50 105	159.5
1	3	5211.0	263.3	48 343	153.9
2	6	4397.0	278.4	47 210	150.3
3	9	3538.1	294.2	45 916	146.2
4	12	2630.8	310.6	44 395	141.3

Table 9: Five steps of the loop, with $d \leftarrow d + r\,\Delta t$.

At each step the grain receives d and returns S; the rest (pressure, thrust) belongs to the ballistic model of HERMES.

Consistency checks.

- *Area at $d = 0$.* By hand, each fin removes outside the core an area $w\,l + w^3/(24R_c)$, so $A_{\text{face}}(0) \approx \frac{\pi}{4}(D_g^2 - D_c^2) - N_f\big(wl + \frac{w^3}{24R_c}\big) = 5989$ mm^2. The code gives 5983.6 mm^2 ($-0.09\,\%$).

- *Area and perimeter.* Since $|\nabla D| = 1$, a front that advances δ sweeps an area $P\,\delta$, so $dA_{\text{face}}/dd = -P$. Between $d = 0$ and 10 mm, the area lost is 2742.5 mm^2 and the integral of P is 2737.6 mm^2 (difference $0.18\,\%$).

- *Volume and surface.* The propellant that burns is the surface times the regression, $dV/dd = -S$. Between $d = 0$ and 10 mm, the volume lost is 476 191 mm^3 and the integral of S is 475 499 mm^3 (difference $0.15\,\%$).

5 Annexes

5.1 Annex A: reference pseudocode

This is the complete procedure, with the same conventions as OpenMotor. Python-like pseudocode; d is in metres.

```
SETUP (once per set of parameters)
  x, y  = meshgrid(linspace(-1,1,M), linspace(-1,1,M))
  mask  = (x^2 + y^2 > 1)                  # outside the grain
  img   = ones(M, M)                       # 1 = propellant, 0 = void
  rc, wn, ln = Dc/Dg, 2*w/Dg, 2*l/Dg       # normalised lengths
  img[x^2 + y^2 < rc^2] = 0                # core
  if inverted: s0, s1, val = rc - ln, rc, 1
  else:        s0, s1, val = 0, rc + ln, 0
  for i in 0 .. Nf-1:
      th = 2*pi*i/Nf
      t  = cos(th)*x + sin(th)*y           # across the fin
      s  = sin(th)*x - cos(th)*y           # along the fin
      img[(abs(t) < wn/2) & (s > s0) & (s < s1)] = val
  Dt    = skfmm.distance(masked(img, mask), dx=1/M) * 2
  Dmax  = max(Dt)
  wmax  = Dmax * Dg/2
  K     = int(Dmax*M) + 2
  dk    = [k/M for k in 0 .. K-1]
  A_k   = [(Dg/M)^2 * count(Dt > dk[k], ignoring masked pixels) for k]
  A_k   = savgol_filter(A_k, 31, 5)
  A_of  = linear_interpolation(dk, A_k)

QUERIES (d in metres)
  face_area(d):   dn = 2*d/Dg
```

```
                if int(dn*M) >= K-1: return 0
                return A_of(dn)
perimeter(d):   length of the contour {Dt = dn}, in pixels, times Dg/M
                # PENDING: see Annex C
ends(d):        (z_top, z_bot) according to the table of Section 4
Lreg(d)     = z_bot - z_top
S(d)        = perimeter(d)*Lreg(d) + n_faces*face_area(d)
V(d)        = face_area(d)*Lreg(d)
A_port(d)   = pi*Dg^2/4 - face_area(d)
web(d)      = wmax - d                    if both ends inhibited
            = min(Lreg(d), wmax - d)      otherwise
V_free(d)   = pi*Dg^2/4*L - V(d)
```

5.2 Annex B: peculiarities found in the OpenMotor code

They do not change the method, but whoever implements it should know them.

- The docstring of `generateCoreMap` says that 0 is propellant and 1 is void. The code does the opposite: 1 is propellant.
- The first fin points along $-y$ and not along $+x$ (rotation of 90° of the whole pattern; no effect on the results).
- The pixel spacing is $2/(M-1)$, but the code uses $2/M$ for distances and D_g/M for areas, which underestimates lengths by about $1/M$ and areas by about $2/M$ (estimated from the code, not measured). **[Decision HERMES]**: keep it to reproduce OpenMotor, or use $M-1$.
- web(d) uses $L(d) = L - 2d$ for the axial term, but the remaining axial distance is $L(d)/2$ when both ends burn. It only matters for short grains, where the axial burn ends first, and the burnout instant is almost unaffected (our reading of the code).
- Check 4 of Table 4 says "less than or equal" but rejects equality.
- **Not covered:** erosive burning, and the mass flux and Mach number in the port, which OpenMotor only uses to raise warnings.

References

[1] J. A. Sethian, A fast marching level set method for monotonically advancing fronts, *Proc. Natl. Acad. Sci. USA* 93(4), 1591–1595, 1996.

[2] J. N. Tsitsiklis, Efficient algorithms for globally optimal trajectories, *IEEE Trans. Autom. Control* 40(9), 1528–1538, 1995.

[3] scikit-fmm documentation, https://scikit-fmm.readthedocs.io.

[4] OpenMotor repository, https://github.com/reilleya/openMotor.

[5] G. P. Sutton and O. Biblarz, *Rocket Propulsion Elements*, Wiley.

[6] R. Nakka, *Solid Rocket Motor Theory*, https://www.nakka-rocketry.net.